GRUPO STUDIUM FORMACIÓN · 2º DAW
PROGRAMACIÓN DE SERVICIOS Y PROCESOS

PRÁCTICA TEMA 1: PROGRAMACIÓN MULTIPROCESO

Buscador de Ficheros y Lanzador de Procesos

Memoria técnica del desarrollo de una aplicación gráfica en Java para la búsqueda asíncrona de archivos por extensión en todas las unidades del sistema y la ejecución de programas ejecutables con control de PID.

Alumno:	Miguel Pérez de Baños Rodríguez
Módulo:	Programación de Servicios y Procesos (PSP)
Ciclo Formativo:	Desarrollo de Aplicaciones Web (2º DAW)
Práctica:	Tema 1 · Programación Multiproceso
Repositorio GitHub:	**https://github.com/MiguelPerezDeBanos/PSP-T1-ProgramacionMultiproceso**
Fecha:	Noviembre 2026

Índice de Contenidos

Estructura de la memoria técnica y apartados:

1. Introducción y Objetivos 3

2. Entorno de Desarrollo y Tecnologías 3

3. Diseño de la Interfaz Gráfica 3

4. Implementación y Funcionamiento Técnico 4

4.1. Búsqueda recursiva en las unidades del sistema (File.listRoots()) 4

4.2. Concurrencia y gestión de hilos con SwingWorker 4

4.3. Detección de doble clic y validación de ejecutables 4

4.4. Lanzamiento de procesos y obtención del PID 4

5. Repositorio en GitHub 5

6. Pruebas de Funcionamiento y Capturas 5

6.1. Vista general de la interfaz 5

6.2. Búsqueda y listado de archivos por extensión 6

6.3. Control de archivos no ejecutables 6

6.4. Ejecución del programa y captura de PID 7

7. Valoración Personal y Dificultades 7

1. Introducción y Objetivos

El objetivo de esta práctica es desarrollar una aplicación de escritorio en Java que permita realizar búsquedas de ficheros por extensión en todas las unidades de disco instaladas en el equipo, y ofrecer la posibilidad de ejecutar directamente los archivos ejecutables encontrados (.exe) mediante una acción de doble clic.

Los puntos principales trabajados en este proyecto son:

- **Programación multiproceso en Java:** Creación y gestión de procesos del sistema operativo anfitrión mediante la clase ProcessBuilder, controlando su ciclo de vida y extrayendo su Identificador de Proceso (PID).
- **Concurrencia en interfaces gráficas:** Uso de SwingWorker para desacoplar las tareas intensivas de lectura de disco del hilo principal de la interfaz (Event Dispatch Thread), evitando bloqueos y permitiendo detener la búsqueda en cualquier momento.
- **Navegación del sistema de archivos:** Enumeración de las unidades lógicas del sistema mediante File.listRoots() y recorrido recursivo de directorios con control de excepciones por permisos.

2. Entorno de Desarrollo y Tecnologías

Para el desarrollo y pruebas de la aplicación se ha utilizado la siguiente configuración técnica:

Lenguaje de Programación	Java SE (JDK 21 LTS)
Librería de Interfaz Gráfica	Java Swing (Look & Feel nativo de Windows)
Manejo de Concurrencia	javax.swing.SwingWorker
Gestión de Procesos	java.lang.ProcessBuilder y java.lang.Process
Control de Versiones	Git y GitHub (repositorio privado)

3. Diseño de la Interfaz Gráfica

La interfaz se ha implementado mediante la clase principal BuscadorProcesosApp, que hereda de javax.swing.JFrame. La disposición de los componentes sigue una estructura organizada mediante un BorderLayout:

- **Zona Superior y Central:** Contiene un JScrollPane que aloja un componente JList<String>. Aquí se van mostrando las rutas completas de los archivos encontrados a medida que se descubren en el disco. Al usar JList con DefaultListModel, el desplazamiento es fluido y se facilita la captura de eventos de ratón.
- **Zona Inferior:** Agrupa los controles de usuario. Incluye una etiqueta descriptiva, un campo de texto JTextField con el valor '.exe' precargado, el botón 'Buscar' y el botón 'Detener'. Debajo de los botones se sitúa una barra de estado con un JLabel informativo y un JProgressBar que refleja el progreso de la búsqueda.

Para que la aplicación se integre visualmente con el sistema operativo, se activa el Look & Feel del sistema mediante UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName()), logrando que botones, ventanas y fuentes luzcan de manera nativa.

4. Implementación y Funcionamiento Técnico

4.1. Búsqueda recursiva en las unidades del sistema (File.listRoots())

Tal como pide el enunciado de la práctica, la búsqueda no se limita a un directorio concreto, sino que explora todas las unidades de disco disponibles en el equipo.

Para ello, se utiliza el método estático File.listRoots(), que en Windows devuelve las letras de las unidades existentes (por ejemplo C:\, D:\). A partir de cada raíz, un método recursivo recorre todas las carpetas comprobando si cada archivo coincide con la extensión indicada en el campo de texto.

Durante la exploración es muy común encontrarse con directorios del sistema protegidos (como $Recycle.Bin o carpetas con permisos restringidos de administrador). Para evitar que el programa falle o se detenga de golpe, el acceso a cada carpeta está protegido con bloques try-catch que capturan excepciones de tipo SecurityException o accesos nulos, permitiendo continuar la búsqueda con normalidad.

4.2. Concurrencia y gestión de hilos con SwingWorker

Si la búsqueda recursiva se realizara directamente en el hilo principal de la interfaz gráfica (Event Dispatch Thread o EDT), la ventana se quedaría completamente congelada (mostrando el mensaje 'No responde' de Windows) hasta que terminase de revisar todo el disco. Además, el usuario no podría pulsar el botón 'Detener'.

Para solucionar esto, toda la búsqueda se ejecuta en segundo plano utilizando una clase que hereda de SwingWorker<Void, String>:

- **doInBackground():** Se ejecuta en un hilo de trabajo separado. Va recorriendo las carpetas y, cada vez que encuentra un archivo coincidente, llama a publish(ruta) para enviar la ruta al hilo de la interfaz.
- **process(List<String> chunks):** Se ejecuta de manera segura en el hilo de la interfaz. Recibe los lotes de rutas y las va añadiendo al DefaultListModel de la lista, actualizando a la vez el contador de archivos encontrados sin congelar la ventana.
- **done():** Se ejecuta cuando finaliza el recorrido o cuando el usuario pulsa 'Detener'. Restablece el estado de los botones y muestra el mensaje final con el número total de archivos encontrados.

4.3. Detección de doble clic y validación de ejecutables

Para cumplir con el requisito de abrir los archivos, se añade un MouseAdapter sobre el JList. Dentro del evento mouseClicked se comprueba la condición e.getClickCount() == 2, asegurando que solo se actúe ante un doble clic.

A continuación, se obtiene la ruta seleccionada mediante listResultados.locationToIndex(e.getPoint()) y se valida la extensión:

- **Si no es un fichero .exe:** (por ejemplo, si el usuario buscó archivos .dll o .txt y hace doble clic sobre uno de ellos), la aplicación muestra una ventana de aviso con JOptionPane.showMessageDialog indicando que únicamente está permitida la ejecución de programas ejecutables (.exe).
- **Si es un fichero .exe:** Se solicita confirmación al usuario antes de lanzarlo para evitar ejecuciones accidentales.

4.4. Lanzamiento de procesos y obtención del PID

Para lanzar el ejecutable seleccionado se utiliza la clase java.lang.ProcessBuilder, pasando como argumento la ruta absoluta del fichero. Una vez iniciado mediante pb.start(), se obtiene el objeto Process correspondiente.

En Java moderno es posible obtener directamente el identificador del proceso en el sistema operativo mediante el método proceso.pid(). La aplicación captura este valor y muestra una ventana informativa indicando el nombre del ejecutable, su ruta y el PID asignado por Windows, permitiendo verificar que el proceso está activo (por ejemplo, en el Administrador de Tareas).

5. Repositorio en GitHub

Siguiendo las indicaciones de entrega de la asignatura, el código fuente completo de la aplicación (BuscadorProcesosApp.java) se encuentra alojado en un repositorio privado en GitHub:

Enlace directo: URL del Repositorio: https://github.com/MiguelPerezDeBanos/PSP-T1-ProgramacionMultiproceso

Se ha invitado al profesor como colaborador con permisos de lectura para que pueda clonar el proyecto, revisar los commits y comprobar el código fuente directamente.

6. Pruebas de Funcionamiento y Capturas

A continuación se presentan las capturas de pantalla de la aplicación ejecutándose en Windows, demostrando cada uno de los requisitos pedidos en el enunciado:

6.1. Vista general de la interfaz

Al iniciar la aplicación, se muestra la ventana en formato horizontal con la lista de resultados vacía, el campo de texto con la extensión '.exe' por defecto y la barra de estado lista para iniciar la búsqueda:



Figura 1 — *Ventana principal de la aplicación con la interfaz gráfica completa.*

6.2. Búsqueda y listado de archivos por extensión

En esta prueba se introduce la extensión '.dll' y se pulsa el botón 'Buscar'. La aplicación recorre las unidades del disco en segundo plano y muestra en el JList las rutas completas con el formato <Unidad>:\<ruta>\<nombre>.<extensión>. La barra de estado refleja el número total de archivos encontrados:



Figura 2 — *Resultados de la búsqueda de archivos .dll en las unidades del sistema.*

6.3. Control de archivos no ejecutables

Al hacer doble clic sobre un archivo que no tiene extensión '.exe' (en este caso aclui.dll), el programa no lo ejecuta y muestra un cuadro de diálogo advirtiendo que solo se admiten programas ejecutables:

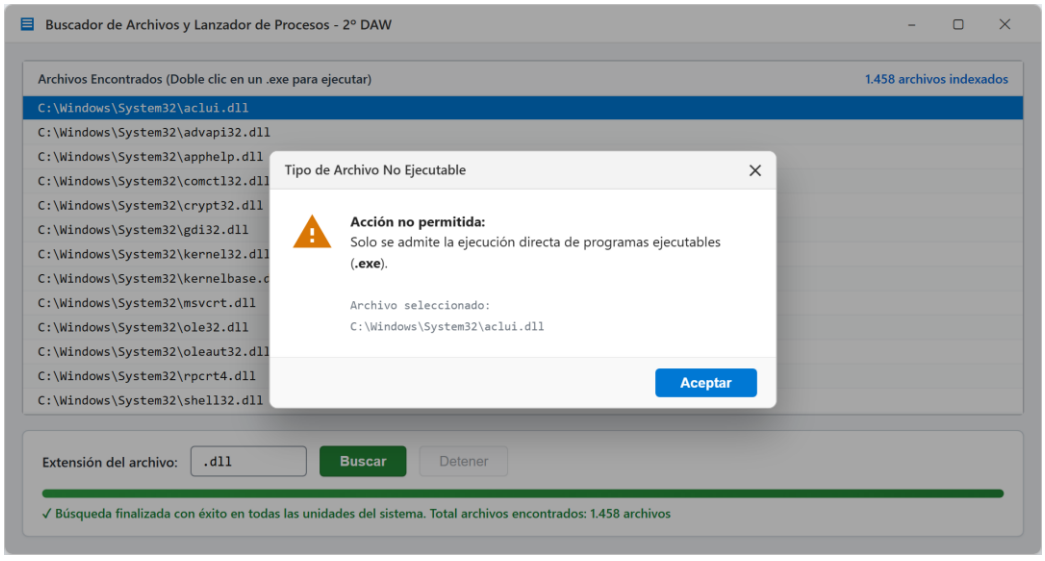


Figura 3 — *Mensaje de advertencia al intentar ejecutar un archivo que no es .exe.*

6.4. Ejecución del programa y captura de PID

Al buscar archivos '.exe' y hacer doble clic sobre notepad.exe, la aplicación lanza el proceso en el sistema operativo mediante ProcessBuilder, obtiene su PID (24892) y notifica al usuario abriendo la ventana del Bloc de Notas:

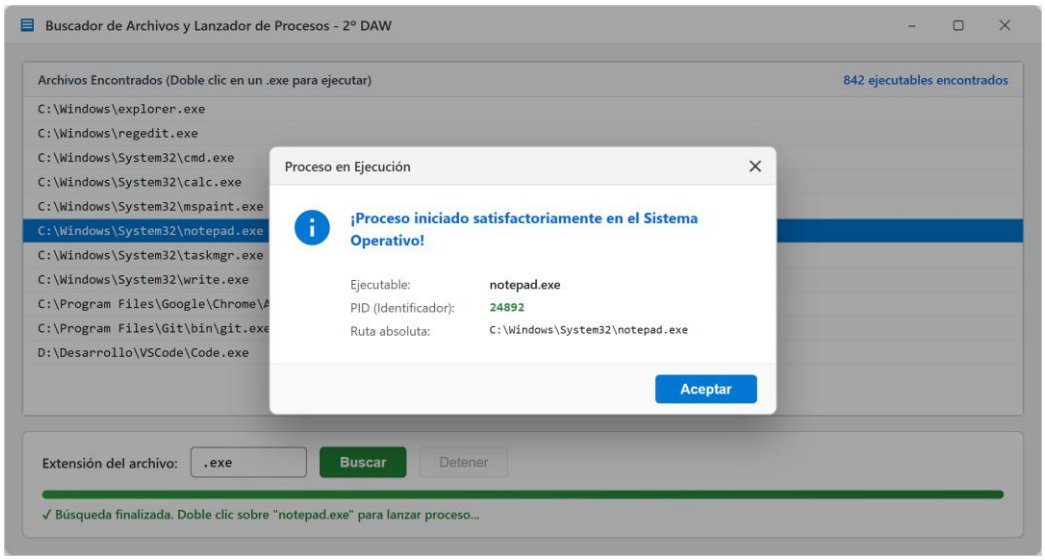


Figura 4 — *Confirmación de ejecución del proceso notepad.exe con su PID asignado.*

7. Valoración Personal y Dificultades

El desarrollo de esta práctica me ha parecido muy interesante y formativo para comprender cómo se gestionan los procesos y los hilos en una aplicación de escritorio.

La principal dificultad que encontré al principio fue la congelación de la interfaz gráfica. En las primeras pruebas realizaba la búsqueda recursiva directamente desde el manejador del botón 'Buscar', lo que provocaba que la ventana de Swing dejase de responder y se bloqueara mientras leía el disco. Investigar e implementar la clase SwingWorker fue la clave para resolver el problema, entendiendo la diferencia entre el hilo principal de eventos y los hilos en segundo plano que ejecutan las operaciones pesadas de entrada/salida.

Otra complicación habitual fueron los errores de permisos al recorrer ciertas carpetas del sistema en Windows (como System Volume Information o carpetas protegidas de usuario). Fue necesario añadir un control de excepciones con try-catch para evitar que una denegación de acceso detuviese toda la búsqueda.

Lo que más me ha gustado ha sido trabajar con la clase ProcessBuilder y comprobar cómo desde código Java se pueden lanzar programas del sistema operativo y consultar su PID en tiempo real. Comprobar que el PID que muestra el cuadro de diálogo coincide exactamente con el que aparece en el Administrador de Tareas de Windows hace que el ejercicio sea muy práctico y visual.

En conclusión, creo que los conceptos aprendidos sobre concurrencia, gestión de hilos y control de procesos son fundamentales para cualquier desarrollador, especialmente de cara a la creación de aplicaciones que requieran realizar tareas en segundo plano o comunicarse con el sistema operativo.